SMARTSTAY AI

AI-Enabled Smart Hostel Management System

A Project Based Learning Report Submitted in partial fulfillment of the course requirements for the award of grades in the subject of

SOA PROGRAMMING AND MICROSERVICES (24SDCS03A)

Submitted by

Abhinaya Mohita 2410030121
Anugoju Srija 2410030349
Cherishma Priya 2410030440
Lakshmi Sri 2410030279
S. Amrutha 2410030278
Sruthika 2410030260

Under the guidance of

Dr. N Rama Rao
Assistant Professor

Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar
Aziz Nagar – 500075

OCTOBER – 2026

ABSTRACT

SmartStay AI is a web-based hostel management system designed to support student accommodation workflows through a centralized application. The project source includes a React and TypeScript frontend and a FastAPI backend that exposes REST endpoints for authentication, student records, room management, and payment records. PostgreSQL is used for persistent data storage through SQLAlchemy models and database sessions.

The backend includes registration and login endpoints, token-based authentication, role-aware access control, student profile management, room-related services, and a payment data model with payment methods and transaction statuses. The payment model supports UPI, QR, and card method labels, along with pending, successful, and failed statuses. In the current project context, payment handling is intended for demonstration; it must not be represented as a real payment gateway unless a provider is separately integrated and tested.

The project provides a foundation for improving hostel administration by organizing student and room information and creating a path for future additions such as payment workflow integration, analytics, notifications, and AI-assisted services. The report describes the features supported by the supplied source code and identifies unfinished integrations as future work.

LIST OF FIGURES

S. No.	Figure No.	Figure Name	Page No.
1	3.1.1	SmartStay AI system architecture	9
2	4.1.1	Application landing / login interface	11
3	4.1.2	Student dashboard interface	12
4	4.1.3	Student management interface	12
5	4.1.4	Room management interface	13
6	4.1.5	Payment module concept / interface	13

Note: Replace or supplement these figure entries with actual screenshots from the final running application before submission.

LIST OF TABLES

S. No.	Table No.	Table Name	Page No.
1	1	Technology stack	8
2	2	Core backend data entities	10
3	3	API endpoint summary	10
4	4	Functional test plan	14
5	5	Limitations and future enhancements	16

TABLE OF CONTENTS

S. No.	Documentation	Page No.
1	Abstract	2
2	List of Figures	3
3	List of Tables	4
4	Introduction	6
5	Literature Survey	7
6	Methodology	9
7	Results	11
8	Conclusion and Future Work	16
9	References	18

INTRODUCTION

1.1 Introduction to Full Stack Development

Full stack development combines the user-facing frontend, backend application logic, and database layer. SmartStay AI follows this structure: the React/TypeScript frontend presents application screens, FastAPI provides HTTP endpoints, and PostgreSQL stores application records.

1.2 Importance of the Project

Hostel administration involves maintaining student information, room allocations, and fee-related records. A centralized web application can reduce duplicated records, make information easier to retrieve, and provide a common interface for students and administrators.

1.3 Motivation

The project aims to organize hostel-related workflows in one system and provide a foundation for future intelligent services. The project is designed as a learning and demonstration application, with clear separation between frontend components, backend routes, data models, and database access.

1.4 Objectives

The objectives are to implement user registration and login; enforce role-based access to protected operations; manage student and room records; provide structured payment records and statuses; connect the frontend to REST APIs; and establish a foundation for future analytics, notification, and AI modules.

1.5 Scope and Limitations

The supplied source includes authentication, student and room modules, SQLAlchemy models, and payment API/model code. The archive does not establish that real payment processing, AI/ML prediction, Kafka messaging, or production deployment is complete. These are treated as planned extensions rather than completed capabilities.

LITERATURE SURVEY

2.1 Existing Systems

Hostel and accommodation systems commonly centralize resident profiles, room assignments, maintenance requests, and fee information. Manual spreadsheets and disconnected forms can lead to inconsistent records and slow retrieval. Web-based systems improve access to shared records, provided authentication and authorization are implemented correctly.

2.2 Web Application Architecture

A frontend-backend-database architecture separates presentation, business logic, and persistence. React supports reusable interactive interfaces, FastAPI supports typed API routes and request validation, and PostgreSQL provides relational constraints and transactions. SQLAlchemy provides an object-relational mapping layer between Python models and database tables.

2.3 Security Considerations

Authentication confirms a user's identity, while authorization controls which operations that user may perform. The project distinguishes student and administrator roles and protects student creation with an admin dependency. Future work should include security testing, token lifecycle improvements, logging, and deployment configuration reviews.

2.4 Identified Gap

The project creates a base for unified hostel workflows but still requires end-to-end integration and validation of several features. In particular, student accounts need corresponding student profiles, and payment status changes must be connected to a controlled demo workflow or a legitimate payment provider.

METHODOLOGY

3.1 Project Type

This is an application-development project that combines a web frontend, REST API backend, and relational database. Implementation is modular, with separate routers, schemas, services, models, and frontend pages.

3.2 Tools and Technologies Used

Frontend: React, TypeScript, CSS, and Vite. Backend: Python and FastAPI. Database: PostgreSQL. Data access: SQLAlchemy. Validation and settings: Pydantic and Pydantic Settings. Authentication: password hashing and bearer-token authentication. Development tools include VS Code, a Python virtual environment, pgAdmin, and the FastAPI Swagger documentation interface.

3.3 System Architecture

The frontend sends HTTP requests to FastAPI endpoints. Authentication endpoints register users and issue access tokens. Protected endpoints use the current-user dependency and role checks. Router functions call service functions or query SQLAlchemy models through a database session. PostgreSQL stores user, student, room, and payment records.

3.4 Module Description

Authentication module: registration, login, current-user details, and token generation. Student module: student record creation for administrators and student-specific reads. Room module: room-related operations. Payment module: create and list payment records for the authenticated student's profile, with method, amount, reference, timestamp, and status fields. Frontend module: user-facing screens and API interaction.

3.5 Data Model

The User entity stores name, email, password hash, role, and creation time. The Student entity stores student ID, profile details, room number, status, and timestamps. The Payment entity references a student and stores amount, method, status, unique reference, and creation time. The room model is also included in the application. Foreign keys and unique constraints support data integrity.

3.6 Implementation Steps

The application configures environment-based settings and a PostgreSQL engine; defines SQLAlchemy models and Pydantic schemas; implements authentication and protected routers; includes the routers in the FastAPI application; initializes model metadata at startup; and provides a React frontend. End-to-end testing should verify that the frontend uses the correct backend URL and that the authenticated user has a matching student profile.

RESULTS

4.1 Implemented Backend Features

The supplied backend source includes registration and login routes, a current-user endpoint, student management routes with role-aware access, room routes, and payment create/list routes. The payment model defines UPI, QR, and CARD methods and PENDING, SUCCESS, and FAILED statuses.

4.2 API Endpoint Summary

POST /api/auth/register registers a user. POST /api/auth/login authenticates credentials and returns a bearer token. GET /api/auth/me returns the current user. GET /api/students lists students according to role; POST /api/students is admin-only. The room router exposes room-related endpoints. POST /api/payments/ creates a pending payment for the current user's student profile; GET /api/payments/ lists that user's payment records.

4.3 Database and Profile Dependency

The payment endpoints locate the Student row by matching the authenticated user's email. If no matching student record exists, the endpoint returns a not-found error. The development checks shared in the project conversation showed GET /api/students returning an empty list and student creation returning a 403 response for a student-role account. This indicates that a matching student profile must be created through the authorized administrative workflow before payment endpoints can work for that account.

4.4 Testing Plan

Tests should cover valid registration, duplicate email rejection, successful and failed login, unauthorized access, student access restrictions, admin-only student creation, payment creation with a valid student profile, payment history retrieval, invalid amount validation, and database persistence after restart. Frontend tests should verify loading, error messages, status display, and refresh of transaction history.

4.5 Testing Status

The source archive supports documentation of the implemented routes and models, but it does not provide sufficient evidence to claim that all frontend-backend flows pass. Record actual test outcomes and add screenshots from the running application before final academic submission.

4.6 Demo Payment Safety

The payment data model supports payment methods and status values, but the supplied code should not be described as a real UPI or card payment gateway. A college demonstration can simulate success or failure and save the resulting status. Real payments require a compliant payment provider, server-side verification, and secure handling of provider callbacks.

CONCLUSION AND FUTURE WORK

5.1 Conclusion

SmartStay AI establishes a modular foundation for hostel management using React/TypeScript, FastAPI, and PostgreSQL. The backend separates authentication, data schemas, routes, services, and models. Its student and payment relationship also demonstrates the importance of linking an authenticated account to a domain-specific student profile.

5.2 Key Findings

Role-based authorization is necessary for administrative operations. Payment records depend on a valid student profile. A working API alone does not prove that the frontend is fully integrated; end-to-end tests are needed to confirm request handling, persisted data, and user-visible status updates.

5.3 Limitations

The current source archive does not demonstrate a complete real-money payment integration or verified completion of AI/ML analytics, Kafka notifications, Docker deployment, or production hosting. Some frontend flows and database initialization should be tested in the target environment.

5.4 Future Enhancements

Add a complete student onboarding workflow; connect payment screens to backend APIs; implement clearly labelled simulated payment outcomes for demonstrations; integrate a payment gateway only if required and approved; add maintenance complaint classification and occupancy analytics; introduce Kafka-based notifications; containerize the application with Docker; add automated tests and CI; and deploy with environment secrets managed securely.

REFERENCES

[1] React Documentation. https://react.dev/

[2] FastAPI Documentation. https://fastapi.tiangolo.com/

[3] PostgreSQL Documentation. https://www.postgresql.org/docs/

[4] SQLAlchemy Documentation. https://docs.sqlalchemy.org/

[5] Pydantic Documentation. https://docs.pydantic.dev/

[6] MDN Web Docs, HTTP and web application fundamentals. https://developer.mozilla.org/

[7] OWASP Foundation, Application Security Verification Standard and authentication guidance. https://owasp.org/

[8] Koneru Lakshmaiah Education Foundation, Project Based Learning documentation template supplied for formatting reference.